

Telemetry-Driven Microservices Orchestration in Cloud-Edge Environments

Angelo Marchese

Dept. of Electrical Electronic and Computer Engineering
University of Catania
Catania, Italy
angelo.marchese@phd.unict.it

Orazio Tomarchio

Dept. of Electrical Electronic and Computer Engineering
University of Catania
Catania, Italy
orazio.tomarchio@unict.it

Abstract—The orchestration of distributed microservices-based applications, particularly within geo-distributed Cloud-to-Edge environments, poses significant challenges. While Kubernetes stands as the predominant container orchestration standard in Cloud data centers, its static container scheduling approach presents limitations in deploying complex, distributed microservices-based applications across Edge environments. Presently, the scheduling process in Kubernetes fails to consider current infrastructure network conditions, resource usage, or runtime application state—crucial factors for mitigating the heterogeneous and dynamic nature of Cloud-to-Edge infrastructure and optimizing application response times.

In this study, we propose an enhancement of the Kubernetes platform by implementing a load and network-aware microservices scheduling and orchestration strategy. The idea is to extend the Kubernetes control and scheduling logic with a dynamic orchestration strategy, continuously adapting application placement based on the real-time state of both the infrastructure and the application itself. We evaluate the efficacy of our approach by comparing it with the default Kubernetes orchestration and scheduling strategy.

Index Terms—Microservices applications, Container technology, Kubernetes scheduler, Kubernetes descheduler, Observability, Cloud-to-Edge continuum

I. INTRODUCTION

Microservice architecture is a cloud-native technology that is becoming increasingly popular for building extensive applications designed for deployment in dispersed and resource-constrained Cloud-to-Edge computing settings [1]. The real strength of microservices lies in their loosely connected, autonomously deployable, and scalable features, which facilitate distributed deployment and flexible integration across powerful Cloud data centers to heterogeneous and often constrained Edge nodes. However, orchestrating the deployment of such applications across diverse environments is a challenging task given the heterogeneous nature of the computing resources available in a Cloud-to-Edge continuum scenario, as well as the multiple microservices competing to gain access to these resources [2]–[5].

Kubernetes is a widely used container orchestration platform that provides a robust framework for deploying and managing microservices [6]. However, it has not been designed to work in geo-distributed and heterogeneous clusters such as the aforementioned Cloud-Edge environments [7], [8]. In addition,

Kubernetes’s default static orchestration policy is limited when dealing with complex microservice-based applications that share the same node clusters. Specifically, Kubernetes does not consider the runtime resource usage and communication patterns of microservices when scheduling them, which can lead to increased shared resource interference and network latencies between them, thus reducing application performance in terms of response time. Despite the widespread adoption of microservices and Kubernetes, there is a need for innovative placement algorithms that can exploit the unique features of microservices to improve application performance in shared clusters characterized by dynamic resource usage patterns and ever changing network latencies [9], [10].

To address these issues, leveraging our previous works [11], [12], we propose extending Kubernetes with a telemetry-driven scheduling and descheduling strategy that dynamically orchestrates applications based on the dynamic infrastructure and application states, continuously adjusting their placement to enhance performance. The primary objective is to overcome the constraints of the static Kubernetes orchestration policies and mitigate the effects of Cloud-Edge infrastructure instability on the application response time.

The rest of the paper is organized as follows. Section II provides some background information about the Kubernetes platform and discusses in more detail some of its limitations that motivate our work. In Section III the proposed approach is presented, providing some implementation details of its components, while Section IV provides results of our prototype evaluation in a testbed environment. Section V examines some related works and, finally, Section VI concludes the work.

II. BACKGROUND AND MOTIVATION

A. Kubernetes Scheduler

Kubernetes, a container orchestration platform, automates the lifecycle management of distributed applications deployed on large-scale node clusters [13]. A typical Kubernetes cluster comprises a control plane and a set of worker nodes. The control plane encompasses various management services running within one or more master nodes, while the worker nodes serve as the execution environment for containerized application workloads. In Kubernetes, the fundamental deployment units are *Pods*, each containing one or more containers and

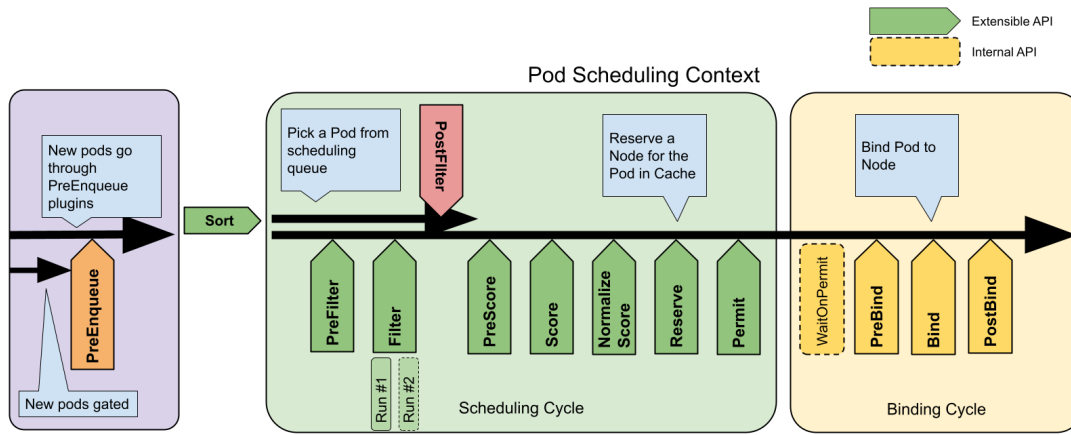


Fig. 1: Kubernetes scheduling framework

managed by *Deployments* resources. In a microservices-based application, each Deployment corresponds to a microservice, and the Pods managed by that Deployment represent individual instances of that microservice.

The *Kube-scheduler*¹ is a control plane component responsible for selecting a cluster node for each Pod, considering both Pod requirements and resource availability on cluster nodes [14]. Every Pod scheduling attempt undergoes a multi-phase process, illustrated in Figure 1, where the sorting, filtering, and scoring phases encompass the primary execution logic. Each phase is implemented by one or more plugins, and these plugins can further implement additional phases. During the sorting phase, an ordering for the Pod scheduling queue is established. In the filtering phase, each plugin executes a filtering function for each cluster node to determine if that node satisfies specific constraints. The output of the filtering phase is a list of candidate nodes that are deemed suitable for running the Pod. In the scoring phase, each plugin executes a scoring function for each candidate node, assigning a score based on specific criteria. The final score for each node is determined by the weighted sum of the individual scores assigned by each scoring plugin. Subsequently, the Pod is assigned to the node with the highest final score. If there are multiple nodes with equal scores, one of them is randomly selected. It's important to note that the Kubernetes scheduler is designed to be extensible. Each scheduling phase serves as an extension point where one or more custom plugins can be registered. The default Kubernetes scheduler employs various in-tree plugins, including the *NodeResourcesFit*, *LeastAllocated*, and *InterPodAffinity* plugins.

The *NodeResourcesFit* plugin handles both the filtering and scoring phases, ensuring that nodes lacking adequate computational resources to satisfy Pod resource requests are filtered out. The plugin assigns scores to nodes based on mutually exclusive strategies:

- *LeastAllocated*: it favors nodes with the lowest ratio

between the weighted sum of requested resources of Pods running on nodes (including the resource requests of the Pod to be scheduled) and the total amount of allocatable resources on those nodes. The objective is to achieve a balanced resource utilization among cluster nodes.

- *MostAllocated*: it favors nodes with the highest ratio between the weighted sum of requested resources of Pods running on nodes (including the resource requests of the Pod to be scheduled) and the total amount of allocatable resources on those nodes. This strategy aims to enhance resource utilization among cluster nodes while concurrently reducing the number of nodes required to efficiently run the workload.
- *RequestedToCapacityRatio*: it distributes Pods to ensure a specified ratio between the sum of requested resources of Pods running on nodes and the total amount of allocatable resources on those nodes.

The *NodeResourcesBalancedAllocation* plugin handles both the filtering and scoring phases and prioritizes nodes that would achieve a more balanced resource usage if the Pod is scheduled there.

The *InterPodAffinity* plugin handles both the filtering and scoring phases and assesses inter-Pod affinity rules, which dictate the placement of Pods based on communication requirements. Each affinity rule signifies a communication relationship between two Pods, indicating they should reside within the same topology domain. A topology domain encompasses either a single node or a group of nodes sharing a common label, specified by the *topologyKey* parameter of the affinity rule. Typically, these nodes are situated within the same Cloud provider region or availability zone. The *weight* parameter of an affinity rule denotes its priority, with higher values indicating a stronger preference for scheduling the associated microservices in close proximity to each other.

However, the default Kubernetes scheduler plugins may not be optimal for orchestrating the placement of complex microservices-based applications within dynamic Cloud-Edge continuum environments. Despite the flexibility offered by the

¹<https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler>

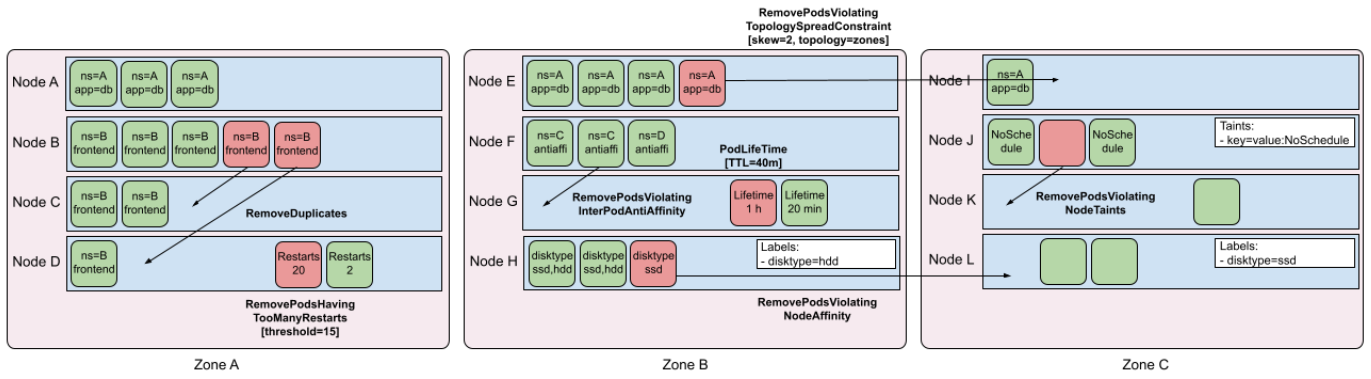


Fig. 2: Kubernetes descheduling strategies

default `NodeResourcesFit` and `NodeResourcesBalancedAllocation` scheduler plugins to define strategies based on different goals, they necessitate knowledge about the resource requirements of each Pod to be scheduled and those already running in the cluster. This information typically comes from resource requests on Pods, which must be specified by application architects before the deployment phase. However, this task is intricate, given that microservices' resource requirements are dynamic parameters strongly dependent on the runtime load and distribution of user requests. Defining Pod resource requirements before the runtime phase can result in inefficient scheduling decisions, subsequently impacting application performance. Overestimating resource requirements for Pods may decrease the resource usage ratio on cluster nodes, leading to increased costs. Conversely, underestimating resource requirements may elevate Pod density on cluster nodes, intensifying shared resource interference and consequently increasing application response times.

Similarly, for the `InterPodAffinity` plugin to effectively prioritize the placement of Pods that frequently communicate with each other on nodes with minimal network distance, it's crucial for application architects to properly define inter-Pod affinity rules and cluster topology domains before the deployment phase. The weight parameters of inter-Pod affinity rules should be aligned with the intensity of communication between Pods, ensuring proportional prioritization. Additionally, nodes should be organized into groups in a manner that nodes with limited network distance are grouped within the same topology domain. However, this task is also complex, as the intensity of communication between Pods in a microservices-based application and the network latency between nodes in a Cloud-Edge environment can change during the runtime phase. Incorrect estimations of these parameters may result in scenarios where Pods with frequent communication are not placed in close proximity to each other, thus leading to increased application response times.

B. Kubernetes Descheduler

Kubernetes scheduler placement decisions are influenced by the cluster state at the time a new Pod appears for scheduling. Given the dynamic nature of Kubernetes clusters and their

evolving state, optimal placement decisions can be enhanced concerning the initial scheduling of Pods. Various reasons may prompt the migration of a Pod from one node to another, including node under-utilization or over-utilization, changes in Pod or node affinity requirements, and events such as node failure or addition.

To achieve this goal, a descheduler component has recently been proposed as a Kubernetes sub-project². This component is responsible for evicting running Pods so that they can be rescheduled onto more suitable nodes. It's important to note that the descheduler itself does not schedule the replacement of evicted Pods; rather, it relies on the default scheduler for that task. The descheduler's policy is configurable and includes default eviction plugins that can be enabled or disabled. It features a common eviction configuration at the top level, along with configuration options from a `Default Evictor` plugin. Figure 2 displays the various default eviction plugins available in the Kubernetes descheduler at the time of writing. Similar to the Kubernetes scheduler, the descheduler is designed to be extensible, allowing integration of custom eviction plugins.

The default Kubernetes descheduler employs various in-tree eviction plugins, including the `LowNodeUtilization` and `HighNodeUtilization` plugins. The `LowNodeUtilization` plugin identifies underutilized nodes and evicts Pods, with the expectation that recreating evicted pods will be scheduled on these underutilized nodes. Conversely, the `HighNodeUtilization` plugin targets nodes with high utilization, evicting Pods in the hope of scheduling them more compactly onto fewer nodes.

It's worth noting that, similar to the `NodeResourcesFit` and `NodeResourcesBalancedAllocation` scheduler plugins, these eviction plugins determine resource usage on each node as the ratio between the weighted sum of requested resources of Pods running on nodes and the total allocatable resources on those nodes. Consequently, the actual resource usage on nodes is not considered, potentially leading to inefficient rescheduling decisions.

²<https://github.com/kubernetes-sigs/descheduler>

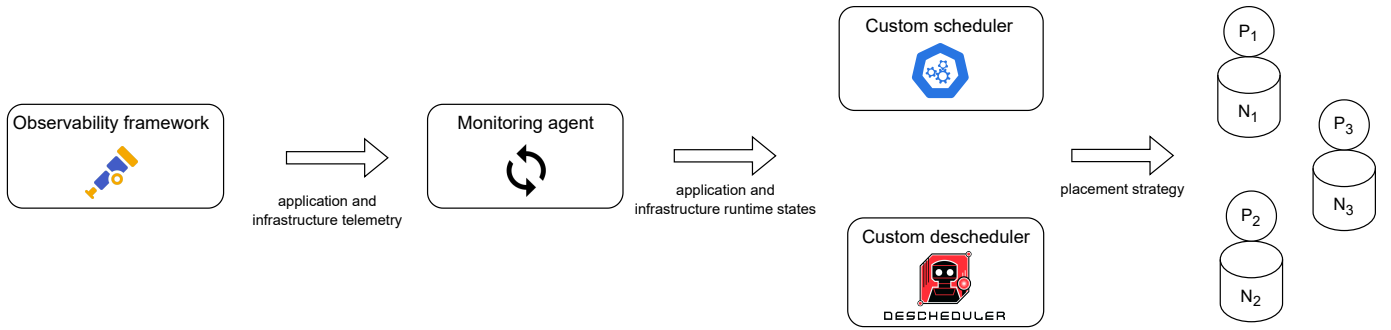


Fig. 3: General model of the proposed approach

III. PROPOSED APPROACH

A. Overall Design

Considering the limitations described in Section II, this work proposes an extension to the Kubernetes platform. The aim is to incorporate a telemetry-driven scheduling and descheduling strategy, rendering the platform suitable for orchestrating complex microservices-based applications with variable resource usage and communication patterns within dynamic Cloud-Edge environments. The primary concept underlying this approach is that the scheduling and descheduling processes for complex microservices-based applications should take into account both the dynamic state of the infrastructure and the real-time resource requirements and communication patterns of the microservices [11], [15]. Although various scheduling and descheduling strategies could be defined based on specific optimization goals, our focus in this work is to minimize runtime shared resource interference among microservices Pods and reduce the network distance between Pods with high intensity of communication. This, in turn, contributes to improve application response time. A notable distinction from the default Kubernetes scheduler and descheduler components lies in our proposed approach's utilization of runtime telemetry data for infrastructure monitoring and microservices resource usage and communication patterns profiling. Unlike statically defined Pod resource requests and inter-Pod affinity rules, this dynamic profiling lessens the burden on application architects to predict resource usage and communication relationships between microservices ahead of time when defining Pod resource requirements and allows for more optimal scheduling and descheduling decisions to be taken.

The general model of the proposed approach is depicted in Figure 3. The continuous monitoring of runtime infrastructure and application microservices resource usage and communication patterns is facilitated through an *observability framework*. This framework comprises a suite of components designed to gather telemetry data, including resource usage for both cluster nodes and Pods, network latencies between nodes, microservices communication traffic, and distributed traces. The *monitoring agent*, leveraging infrastructure telemetry data, annotates each cluster node with resource usage and network latencies information, providing a runtime view of the cluster

state. Similarly, the same agent utilizes application telemetry data to annotate each application microservice Deployment with resource usage and communication relationships information, determining the runtime view of the application state. The *custom scheduler* employs these runtime cluster and application states to determine optimal placements for each application Pod. Meanwhile, the *custom descheduler* is responsible for taking Pod rescheduling actions if better scheduling decisions can be made. Further details regarding the components of the proposed approach are provided in the following subsections.

B. Observability Framework

Given the telemetry-driven approach underlying the proposed scheduling and descheduling strategies, the implementation necessitates an observability framework. This framework serves to gather all essential parameters crucial for establishing a real-time perspective on both the infrastructure and the hosted application. Illustrated in Figure 4 are the principal components of this framework, encompassing elements operating within the Kubernetes control plane as well as those deployed on the worker nodes.

Prometheus³ is a time-series database used to collect metrics from metrics exporters. Jaeger⁴ is a distributed tracing database that collects distributed traces from tracing agents. A trace constitutes a comprehensive log of a distributed transaction, typically initiated by an external user request and involving a chain of interconnected microservices. Within this trace, individual microservice operations or interactions with other microservices are captured as spans. These spans, representing distinct segments of the transaction, are interconnected within the trace through a correlation ID. This ID is generated by the initial microservice engaged in the transaction and is subsequently propagated within the requests to all other microservices involved, facilitating correlation and traceability across the entire transaction flow.

Metrics related to the runtime utilization of CPU, memory, network, and disk bandwidth resources across cluster nodes and Pods are exported to the Prometheus server through the

³<https://prometheus.io>

⁴<https://www.jaegertracing.io>

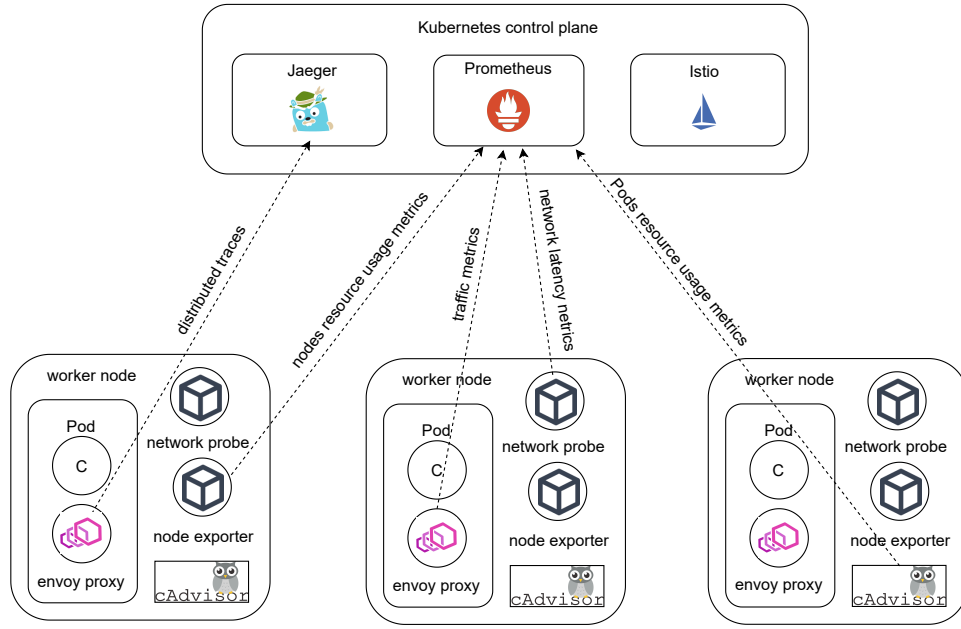


Fig. 4: Observability framework

utilization of *node exporters*⁵ and *cAdvisor*⁶ agents respectively. These agents are executed on each node within the cluster. Network latencies between nodes are measured via network probes by sending ICMP traffic to the other nodes in the cluster.

Furthermore, application traffic metrics and distributed traces are gathered through the Istio service mesh. Within this architecture, the Istio server, executed on the Kubernetes control plane, injects a sidecar container running an Envoy proxy into each Pod upon creation. All traffic between Pods is intercepted by their corresponding Envoy proxies, which expose traffic statistics via metrics exporters that can be queried by the Prometheus server. Additionally, Envoy proxies generate spans for each intercepted request, which are subsequently transmitted to the Jaeger server.

C. Monitoring Agent

The telemetry data exposed by the observability framework, in particular Prometheus metrics and Jaeger distributed traces, is used by the monitoring agent to periodically determine the total runtime utilization of CPU, memory, network, and disk bandwidth resources for both cluster nodes and Pods. Additionally, it determines network latencies between nodes and the communication intensity between Pods.

This component is a Kubernetes operator written in the Java language using the Quarkus Operator SDK⁷ and runs as a Deployment in the Kubernetes control plane. Being a Kubernetes operator, it is triggered by Kubernetes custom resources, specifically the *Cluster* and *AppGroup* custom resources.

A Cluster resource includes a *spec* property with two sub-properties: *runPeriod* and *nodeSelector*. The *runPeriod* property determines the interval in seconds between two consecutive executions of the custom resource processing logic. Meanwhile, the *nodeSelector* property acts as a filter, selecting the list of nodes in the cluster that should be monitored by the agent. During each iteration the list of *Node* resources that satisfy the condition specified by the *nodeSelector* property is fetched from the Kubernetes API server. For each node n_i , the agent assigns annotations like *cpu-usage*, *memory-usage*, *net-usage*, and *disk-usage*, indicating the real-time total utilization of CPU, memory, network, and disk bandwidth resources on that specific node. Following this, for each node n_j , the agent assigns the annotation *network-latency.n_j* to node n_i , with its value corresponding to the runtime network latency between nodes n_i and n_j . Finally the set of nodes with the updated annotations are submitted to the Kubernetes API server.

An AppGroup resource contains a *spec* property with three sub-properties: *runPeriod*, *namespace*, and *deploymentSelector*. The *runPeriod* property determines the interval between two consecutive executions of the custom resource processing logic. The *namespace* and *deploymentSelector* properties are used to select the set of Deployment resources that compose a specific microservices-based application. The Deployments selected by the Application resource are those created in the namespace specified by the *namespace* property and with labels that satisfy the *deploymentSelector* property condition. During each iteration the list of *Deployment* resources selected by the name and namespace properties are fetched from the Kubernetes API server. For each Deployment D_i the agent assigns annotations like *cpu-usage*, *memory-usage*, *net-usage*, and *disk-usage*, indicating the average utilization of CPU, memory, network, and disk bandwidth resources of all

⁵https://github.com/prometheus/node_exporter

⁶<https://github.com/google/cadvisor>

⁷<https://github.com/quarkiverse/quarkus-operator-sdk>

- p : the Pod to be scheduled.
- n : the node to be scored.
- cns : the set of nodes in the cluster, including node n .
- nl : the network latencies between node n and all the other nodes cns , equal to the value of the corresponding *network-latency* annotations on node n .
- aff : the communication affinities between the Pod p and all the other application Pods, equal to the value of the corresponding *aff* annotations on the Deployment owning the Pod p .

The algorithm starts by initializing the variable cmc to zero, representing the total communication cost between Pod p and all other Pods in the application when Pod p is placed on node n . It then iterates through the list of cluster nodes cns . For each cluster node cn , the algorithm calculates the value of $pcmc$, representing the communication cost between Pod p and all other Pods $cn.pods$ currently running on node cn when Pod p is placed on node n . For each Pod $cn.p$ running on node cn , the algorithm multiplies the value of the $aff_{p,cnp}$ parameter by the network latency $nl_{n,cn}$ between nodes n and cn , adding it to the $pcmc$ variable. Subsequently, the value of $pcmc$ is added to the cmc variable. The final node score is the opposite of the cmc variable value. Node scores are then normalized in the range between 0 (*MinNodeScore*) and 100 (*MaxNodeScore*).

The scoring mechanism of the NetworkAware plugin ensures that Pod p is placed on a node, or a nearby node in terms of network latencies, where Pods with which Pod p has the highest communication affinity are executed. This communication affinity is proportional to the volume of traffic exchanged between Pods and the number of distributed traces in which both Pods participate. By considering the current node-to-node network latencies and the dynamic communication relationships between microservices, the proposed NetworkAware plugin implements a dynamic Pod scheduling strategy distinct from the default inter-Pod affinity plugin, which employs a static scheduling strategy.

For each affinity rule, the default Kubernetes scheduler assigns a non-zero score only to nodes belonging to a topology domain matched by the *topologyKey* parameter of the rule. In contrast, the NetworkAware plugin scores all cluster nodes based on their current relative network distances. This enables the implementation of a more finely-grained node scoring approach capable of accommodating the ever-changing network conditions within the cluster, rather than relying on statically assigned node labels before runtime.

Furthermore, unlike the inter-Pod affinity plugin, which considers statically assigned communication affinity weights between Pods, the NetworkAware plugin evaluates the traffic between Pods and the traces in which both are involved to determine the effective communication affinities between them, which may change dynamically at runtime, based on the user request load and distribution.

E. Custom Descheduler

The custom descheduler operates as a CronJob in the Kubernetes control plane and enhances the default Kubernetes descheduler by implementing a custom evictor plugin. This custom plugin is written in the Go language and is based on the Kubernetes descheduler framework.

The descheduler CronJob is configured to periodically run Jobs that execute the descheduling logic, comprising both the default evictor plugin and the proposed custom evictor plugin. Within each iteration, the custom descheduler is configured to evict at maximum one Pod, while ensuring that at least one replica is maintained for each Deployment.

During each execution of the descheduler CronJob, the *Deschedule()* function of the custom eviction plugin is invoked. For each Pod p currently running in the node n_i the function assigns a score $dscore$ determined by Equation (7):

$$dscore(p) = \max_{n_j \in cns} (score(p, n_j)) - score(p, n_i) \quad (7)$$

where $score(p, n_i)$ is the average between the scores assigned by the *Score()* functions of the LoadAware and NetworkAware scheduler plugins and cns is the set of cluster nodes. The Pod p selected for eviction is the one satisfying the conditions:

$$dscore(p) = \max_{p_j \in ps} (dscore(p, n_j)) \quad (8)$$

and

$$dscore(p) > 0 \quad (9)$$

where ps is the set of Pods running in the cluster.

This means that the Pod evicted is the one that would experience the greatest improvement in node score if it were to be rescheduled onto another node with a higher score than the one where it is currently executed. The configuration of the Default evictor plugin ensures that nodes where the Pod cannot be executed due to scheduling constraints are not considered as candidate nodes for migrating the Pod. This prevents the Pod from being unable to be rescheduled after eviction. Similar to the default Kubernetes descheduler, the proposed custom descheduler does not schedule replacements for evicted Pods; instead, it relies on the custom scheduler for that task.

The purpose of the proposed custom descheduler is to provide running Pods the opportunity to be rescheduled based on their runtime resource usage and communication patterns, as well as the cluster network latencies. This approach aims to optimize application placement at runtime. By evicting currently running Pods and subsequently forcing them to be rescheduled, better placement decisions can be taken that help reduce the shared resource interference among Pods and the network distance between those with high intensity of communication, ultimately enhancing the application response time.

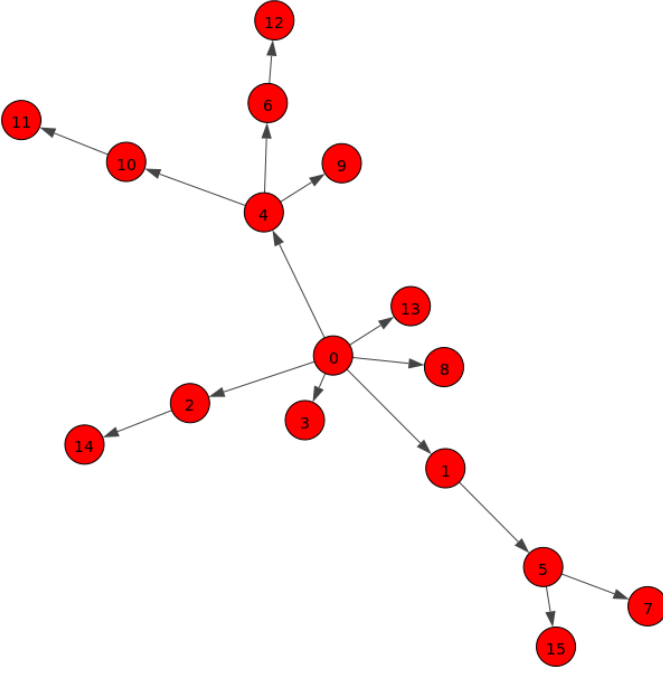


Fig. 5: Sample application topology

IV. EVALUATION

The proposed solution has been evaluated by using a sample microservices-based application generated using the μ Bench benchmarking tool [16]. μ Bench enables the generation of service-mesh topologies with multiple microservices, each running a specific function. Among the pre-built functions in μ Bench, the *Loader* function models a generic workload that stresses node resources when processing HTTP requests. When invoked, the *Loader* function computes an N number of decimals of π . The larger the interval, the greater the complexity and stress on the CPU. Additional stress on node memory and disk bandwidth can be configured by adjusting the amount of memory and the number of disk read and write operations required by the function for each computation. Finally, function network bandwidth usage can be configured by adjusting the number of bytes returned by the function in response to each request.

Figure 5 depicts the sample application topology created using the μ Bench tool. The application comprises sixteen microservices, each running two replicas. Microservice m_0 serves as the entry point for external user requests, which are then handled by backend microservices. These backend microservices interact with each other through the exchange of HTTP requests. The sixteen microservices are grouped into groups of four items, where microservices within the same group run the *Loader* function with the same configuration parameters. Four versions of the *Loader* function are defined, f_0 , f_1 , f_2 and f_3 , each configured to stress a different resource type, in particular CPU, memory, network, and disk bandwidth respectively.

The resource usage configuration parameter for each function are the followings:

- f_0 : 1500 π decimals generated per request.
- f_1 : 200MB of average memory usage.
- f_2 : 20KB of data returned per request.
- f_3 : 20MB of data written on disk per request.

The test bed environment for the experiments consists of a Rancher Kubernetes Engine 2 (RKE2)⁸ Kubernetes cluster with one master node and six worker nodes. These nodes are deployed as virtual machines on a Proxmox⁹ environment and configured with 2 vCPU, 8GB of RAM, a Gigabit virtual network adapter and a virtual disk with read and write bandwidth of around 500 MB/s. In order to simulate a realistic Cloud-to-Edge continuum environment with geo-distributed nodes, network latencies between cluster nodes are simulated by using the Linux traffic control (*tc*)¹⁰ utility. By using this utility network latency delays are configured on virtual network cards of the cluster nodes.

Black box experiments are conducted by evaluating the end-to-end response time of the sample application when HTTP requests are sent to the microservice m_0 with a specified number of virtual users each sending one request every second in parallel. Requests to the application are sent through the k6 load testing utility¹¹ from a node inside the same network where cluster nodes are located. Each experiment consists of 5 trials, during which the k6 tool sends requests to the microservice m_0 for 60 minutes. The trial interval is partitioned into 10 minutes sub-intervals, during which the k6 tool sends requests to a different worker node. By sending user requests to different nodes geo-distribution of users is simulated. For each trial, statistics about the end-to-end application response time are measured and averaged with those of the other trials of the same experiment. For each experiment, we compare both cases when our custom scheduler and descheduler components are deployed on the cluster and when only the Kubernetes scheduler is present with the default configuration. The *runPeriod* properties of the *Cluster* and *AppGroup* custom resources for the collection of infrastructure and telemetry data by the monitoring agent are configured with a value of 10 seconds, while the descheduler logic is executed every 20 seconds. Four different scenarios are considered, based on the simulated network latency between cluster nodes: 10ms, 50ms, 100ms and 200ms.

Figures 6, 7, 8, and 9 depict the outcomes of the four conducted experiments, each representing a distinct scenario. They showcase the 95th percentile of the application response time against the number of virtual users concurrently sending requests to the application. Across all scenarios, the proposed approach consistently outperforms the default Kubernetes scheduler, demonstrating average enhancements of 39%, 41%, 46%, and 53% in the respective scenarios.

⁸<https://docs.rke2.io>

⁹<https://www.proxmox.com>

¹⁰<https://man7.org/linux/man-pages/man8/tc.8.html>

¹¹<https://k6.io>

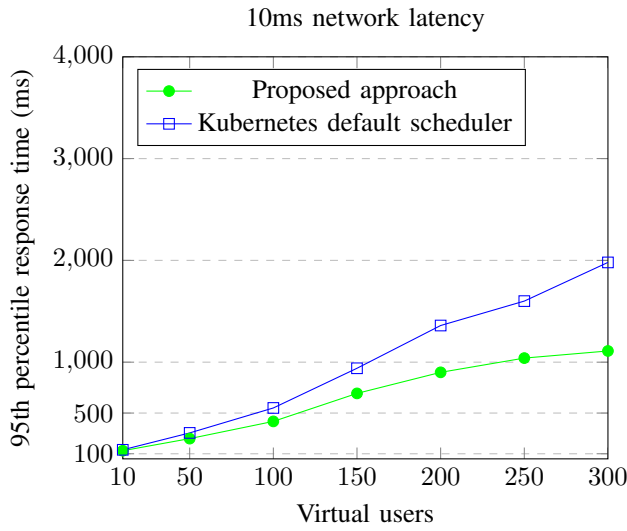


Fig. 6: Experiments results (Scenario 1)

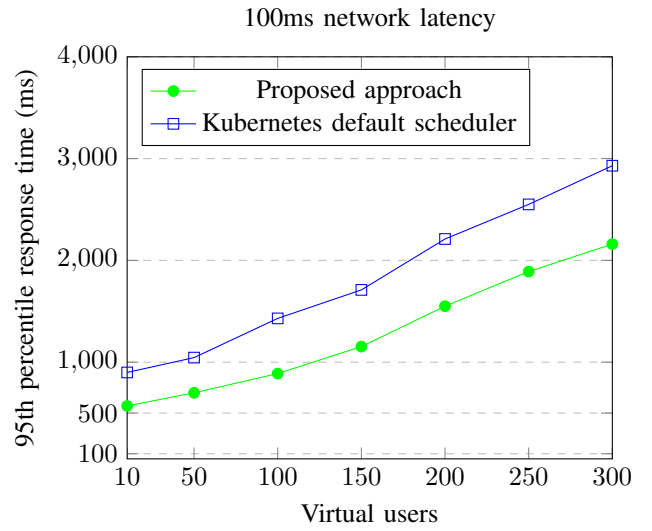


Fig. 8: Experiments results (Scenario 3)

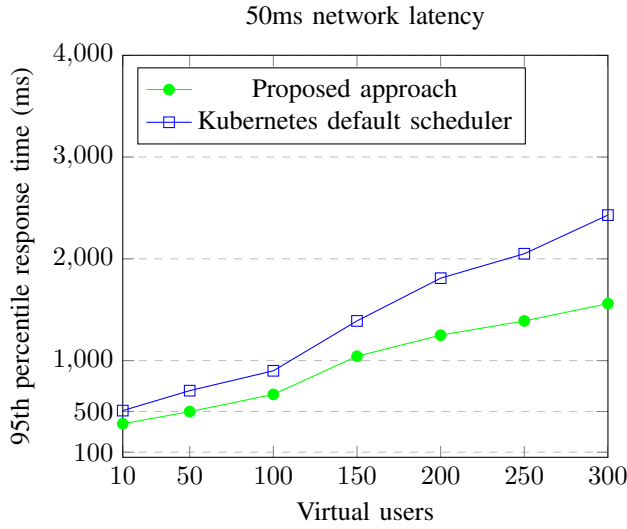


Fig. 7: Experiments results (Scenario 2)

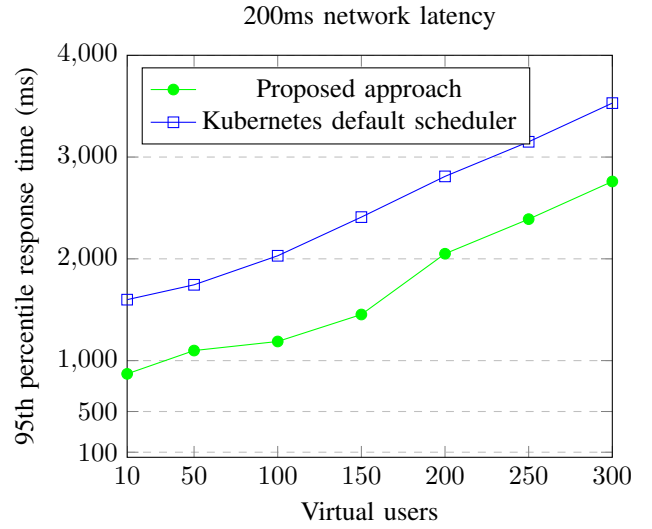


Fig. 9: Experiments results (Scenario 4)

In the initial scenario, the application response time is minimally affected by network communication due to low node-to-node network latencies. Consequently, the proposed network-aware scheduling strategy yields modest improvements in application response time. Moreover, for a limited number of virtual users, the proposed approach exhibits similar performance to the default scheduler, attributed to minimal shared resource interference between Pods, despite being placed on the same nodes by the default scheduler. However, as the number of virtual users increases, the proposed approach surpasses the default scheduler, particularly exhibiting higher performance with higher virtual user counts. The default scheduler's response time increases more rapidly compared to the proposed approach, owing to the load-aware scheduling strategy employed, which allocates Pods to cluster nodes based on their runtime resource usage, thereby mitigating shared resource

interference. In subsequent scenarios, network communication emerges as a bottleneck for application response time. In these scenarios, our approach outperforms the default scheduler even for low numbers of virtual users, with greater enhancements observed for higher node-to-node network latencies. However, an outcome of integrating both load-aware and network-aware scheduling strategies is that as the number of virtual users increases, response time grows more rapidly for higher network latencies, still remaining lower than response times with the default scheduler. This phenomenon is explained by the increased average resource usage of microservices with higher virtual user counts, leading to greater distribution of Pods among cluster nodes due to the load-aware scheduling strategy. Consequently, this increases network latency between application microservices, consequently amplifying end-to-end application response times.

V. RELATED WORK

In the literature, various works propose extending the Kubernetes platform to address the limitations of its static orchestration strategy when applied to microservices-based applications executed on Cloud-Edge environments [14], [17], [18].

In [19] Nautilus is presented, a run-time system that includes, among its modules, a communication-aware microservice mapper. This module divides the microservice graph into multiple partitions based on the communication overhead between microservices and maps the partitions to the cluster nodes in order to make frequent data interaction complete in memory. While the proposed solution migrates application Pod if computational resources utilization is unbalanced among nodes, there is no Pod rescheduling in the case of degradation in the communication between microservices.

NetMARKS [20] is a Kubernetes scheduler extender that uses dynamic network metrics collected with Istio Service Mesh to ensure an efficient placement of Service Function Chains, based on the historical traffic amount exchanged between services. As in our work, historical application traffic is measured, but the proposed scheduler does not take into account current node-to-node latencies to measure their network distance.

In [21] a scheduling framework is proposed which enables edge sensitive and Service-Level Objectives (SLO) aware scheduling in the Cloud-Edge-IoT Continuum. The proposed scheduler extends the base Kubernetes scheduler and makes scheduling decisions based on a service graph, which models application components and their interactions, and a cluster topology graph, which maintains current cluster and infrastructure-specific states. However, this work does not consider historical information about the traffic exchanged between microservices in order to determine their run time communication relationships.

Pogonip, an edge-aware scheduler for Kubernetes, designed for asynchronous microservices is presented in [22]. Authors formulate the placement problem as an Integer Linear Programming optimization problem and define a heuristic to quickly find an approximate solution for real-world execution scenarios. The heuristic is implemented as a set of Kubernetes scheduler plugins. However in this work, there is no Pod rescheduling if network conditions change over time.

In [23] Diktyo is presented, a network-aware scheduling framework for the Kubernetes platform, inspired by its scheduling plugins architecture. Diktyo proposes additional scheduling plugins aiming to determine low-latency deployment schemes for applications scheduled on Kubernetes clusters. The framework minimizes the application's end-to-end latency by selecting nodes with low network costs for dependent microservices and with sufficient bandwidth capacity based on previous microservice allocations. Also in this work, there is no Pod rescheduling if network conditions change over time.

Finally, in [24] DRS is proposed, a deep reinforcement learning enhanced Kubernetes scheduler, to mitigate the re-

source fragmentation and low utilization issues caused by the inefficient policies of the default Kubernetes scheduler. The Kubernetes scheduling problem is modeled as a Markov decision process with designed state, action, and reward structures to increase resource usage and decrease load imbalance. Then, a DRS monitor is designed to perceive parameters concerning resource utilization and create a thorough picture of all available resources globally. Finally, DRS is configured to automatically learn the scheduling policy through interaction with the Kubernetes cluster, without relying on expert knowledge about workload and cluster status. Also in this work, no Pod rescheduling actions are taken to find more optimal scheduling decisions for running Pods.

VI. CONCLUSIONS

In this work we proposed to extend the Kubernetes platform with a telemetry-driven orchestration strategy aimed at minimizing QoS violations on the response times of distributed microservices-based applications executed on dynamic Cloud-Edge environments. The main goal is to overcome the limitations of the Kubernetes static scheduling and descheduling policies that require ahead of time knowledge of computational resource requirements of each microservice and their communication patterns to make optimal container placement and rescheduling decisions. Considering the dynamic nature of distributed microservices applications, the idea is to extend the Kubernetes scheduler and descheduler components with custom plugins that make use of runtime infrastructure and microservices resource usage and network telemetry data to make their decisions. In this way, the effort for static application resource usage and communication relationships profiling can be reduced, while at the same time guaranteeing the expected application performances. As a future work, we plan to improve the proposed custom scheduling and descheduling strategies by using AI and time series analysis techniques in order to design more sophisticated algorithms that take into account long-term telemetry data to improve predictions on both resource usage and network parameters.

ACKNOWLEDGMENT

This work was partially supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, partnership on "Telecommunications of the Future" (PE00000001 - program "RESTART") and by University of Catania through the research project "MANGO - Piaceri 2020-2022 - Linea 2".

REFERENCES

- [1] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud container technologies: A state-of-the-art review," *IEEE Transactions on Cloud Computing*, vol. PP, pp. 1–1, 05 2017.
- [2] W. Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, and A. Ahmed, "Edge computing: A survey," *Future Generation Computer Systems*, vol. 97, pp. 219–235, 2019.
- [3] S. Svorobej, M. Bendeche, F. Griesinger, and J. Domaschka, *Orchestration from the Cloud to the Edge*. Cham: Springer International Publishing, 2020, pp. 61–77.

- [4] A. Ullah, T. Kiss, J. Kovács, F. Tusa, J. Deslauriers, H. Dagdeviren, R. Arjun, and H. Hamzeh, "Orchestration in the cloud-to-things compute continuum: taxonomy, survey and future directions," *J. Cloud Comput.*, vol. 12, no. 135, Sep. 2023.
- [5] D. Calcaterra, G. Di Modica, and O. Tomarchio, "Cloud resource orchestration in the multi-cloud landscape: a systematic review of existing frameworks," *Journal of Cloud Computing*, vol. 9, no. 49, 2020.
- [6] Kubernetes, "Production-Grade Container Orchestration," <https://kubernetes.io>, 2024, last accessed 3 Jun 2024.
- [7] P. Kayal, "Kubernetes in fog computing: Feasibility demonstration, limitations and improvement scope : Invited paper," in *2020 IEEE 6th World Forum on Internet of Things (WF-IoT)*, 2020, pp. 1–6.
- [8] S. Böhm and G. Wirtz, "Towards Orchestration of Cloud-Edge Architectures with Kubernetes," in *Science and Technologies for Smart Cities*, S. Paiva, X. Li, S. I. Lopes, N. Gupta, D. B. Rawat, A. Patel, and H. R. Karimi, Eds., Springer International Publishing, 2022, pp. 207–230.
- [9] M. Goudarzi, M. Palaniswami, and R. Buyya, "Scheduling iot applications in edge and fog computing environments: A taxonomy and future directions," *ACM Comput. Surv.*, vol. 55, no. 7, dec 2022. [Online]. Available: <https://doi.org/10.1145/3544836>
- [10] K. Manaouil and A. Lebre, "Kubernetes and the Edge?" Inria Rennes - Bretagne Atlantique, Research Report RR-9370, Oct. 2020. [Online]. Available: <https://hal.inria.fr/hal-02972686>
- [11] A. Marchese and O. Tomarchio, "Extending the Kubernetes Platform with Network-Aware Scheduling Capabilities," in *Service-Oriented Computing: 20th International Conference, ICSOC 2022, Seville, Spain, November 29 – December 2, 2022, Proceedings*. Seville, Spain: Springer-Verlag, 2022, p. 465–480.
- [12] —, "Application and Infrastructure-Aware Orchestration in the Cloud-to-Edge Continuum," in *IEEE International Conference on Cloud Computing, CLOUD 2023*. Chicago (USA): IEEE, Jul. 2023, pp. 262–271.
- [13] D. Gannon, R. Barga, and N. Sundaresan, "Cloud-native applications," *IEEE Cloud Computing*, vol. 4, pp. 16–21, Sep. 2017.
- [14] C. Carrión, "Kubernetes scheduling: Taxonomy, ongoing issues and challenges," *ACM Comput. Surv.*, vol. 55, no. 7, dec 2022. [Online]. Available: <https://doi.org/10.1145/3539606>
- [15] A. Marchese and O. Tomarchio, "Network-Aware Container Placement in Cloud-Edge Kubernetes Clusters," in *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, Taormina, Italy, May 2022, pp. 859–865.
- [16] A. Detti, L. Funari, and L. Petrucci, "µbench: An open-source factory of benchmark microservice applications," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 3, pp. 968–980, 2023.
- [17] Z. Rejiba and J. Chamanara, "Custom scheduling in kubernetes: A survey on common problems and solution approaches," *ACM Comput. Surv.*, vol. 55, no. 7, dec 2022. [Online]. Available: <https://doi.org/10.1145/3544788>
- [18] K. Senjab, S. Abbas, N. Ahmed, and A. u. R. Khan, "A survey of kubernetes scheduling algorithms," *Journal of Cloud Computing*, vol. 12, no. 1, p. 87, Jun 2023.
- [19] K. Fu, W. Zhang, Q. Chen, D. Zeng, X. Peng, W. Zheng, and M. Guo, "Qos-aware and resource efficient microservice deployment in cloud-edge continuum," in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 932–941.
- [20] L. Wojciechowski, K. Opasiak, J. Latusek, M. Wereski, V. Morales, T. Kim, and M. Hong, "Netmarks: Network metrics-aware kubernetes scheduler powered by service mesh," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, 2021, pp. 1–9.
- [21] S. Nastic, T. Pusztai, A. Morichetta, V. C. Pujol, S. Dustdar, D. Vii, and Y. Xiong, "Polaris scheduler: Edge sensitive and slo aware workload scheduling in cloud-edge-iot clusters," in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*, 2021, pp. 206–216.
- [22] T. Pusztai, F. Rossi, and S. Dustdar, "Pogonip: Scheduling asynchronous applications on the edge," in *IEEE 14th International Conference on Cloud Computing (CLOUD)*, Sep. 2021, pp. 660–670.
- [23] J. Santos, C. Wang, T. Wauters, and F. De Turck, "Diktyo: Network-aware scheduling in container-based clouds," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 4461–4477, 2023.
- [24] Z. Jian, X. Xie, Y. Fang, Y. Jiang, Y. Lu, A. Dash, T. Li, and G. Wang, "Drs: A deep reinforcement learning enhanced kubernetes scheduler for microservice-based system," *Software: Practice and Experience*, vol. n/a, no. n/a, 2023.